

Tema 6: Árboles de búsqueda y maps

6.1 El problema de la búsqueda

Hasta ahora hemos visto dos estructuras lineales principales para guardar datos:

Contenedor	Acceso	Búsqueda	Inserción en medio
<code>vector<T></code> (desordenado)	$\Theta(1)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$
<code>vector<T></code> (ordenado)	$\Theta(1)$	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$
<code>list<T></code>	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\Theta(1)$
<code>map<K, V></code>	—	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
<code>set<T></code>	—	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$

Queda claro: si la búsqueda es la operación predominante, necesitamos una estructura especializada. La solución es un **Árbol Binario de Búsqueda (BST)**.

6.2 Árbol Binario de Búsqueda (BST)

Un **BST** (*Binary Search Tree*) es un `BinTree<T>` que cumple una regla estructural adicional en cada nodo:

Invariante BST: Todos los valores del **subárbol izquierdo** son **estrictamente menores** que el nodo, y todos los del **subárbol derecho** son **estrictamente mayores**.

[Gràfic interactiu disponible a la versió web de l'apunt]

Gracias al invariante, el recorrido **inorden** (*izquierdo* $\rightarrow$ *raíz* $\rightarrow$ *derecho*) visita todos los valores **en orden ascendente** en $\Theta(n)$.

6.3 Búsqueda en un BST

La búsqueda es la operación más directa y eficiente. En cada paso **descartamos la mitad del árbol** sin siquiera explorarla:

```
bool bst_buscar(const BinTree<int>& a, int x) {
    if (a.empty()) return false;
    if (x == a.value()) return true;
    return bst_buscar(a.value() < x ? a.right() : a.left(), x);
}
```

Observa el poder de la propiedad BST: si $x < a.value()$, **sabemos con certeza** que x no puede estar en el subárbol derecho. Lo descartamos completamente sin explorarlo.

[Simulació interactiva disponible a la web]

Nota Principal

En un BST **equilibrado**, la búsqueda cuesta $\mathcal{O}(\log n)$. En el peor caso (árbol degenerado/lista), puede llegar a $\mathcal{O}(n)$. La clave es mantener el árbol balanceado.

Mínimo y Máximo

El valor mínimo es siempre el nodo más a la izquierda (bajamos todo el camino hacia la izquierda hasta encontrar un hijo vacío). El máximo, el más a la derecha.

```
int bst_min(const BinTree<int>& a) {
    assert(!a.empty());
    if (a.left().empty()) return a.value();
    return bst_min(a.left());
}

int bst_max(const BinTree<int>& a) {
    assert(!a.empty());
    if (a.right().empty()) return a.value();
    return bst_max(a.right());
}
```

6.4 Inserción en un BST

Como `BinTree<T>` es **immutable**, no podemos modificar el árbol existente. La inserción **reconstruye el camino** desde la raíz hasta el punto de inserción, reutilizando los subárboles que no cambian:

```
BinTree<int> bst_insertar(const BinTree<int>& a, int x) {
    if (a.empty())
        return BinTree<int>(x);           // Caso base: aquí va el nuevo nodo
    if (x == a.value()) return a;          // Ya existe, no insertamos duplicado
    if (x < a.value())
        return BinTree<int>(a.value(),
                            bst_insertar(a.left(), x), // Reconstruimos rama izq
                            a.right());               // Reutilizamos rama derecha
    return BinTree<int>(a.value(),
                        a.left(),
                        bst_insertar(a.right(), x));   // Reutilizamos rama izq
}
```

[Simulación interactiva disponible a la web]

Fijaos que los subárboles que **no estamos atravesando** se reutilizan directamente sin copiarlos. Gracias a la inmutabilidad funcional de `BinTree`, el compilador se encarga de optimizar la compartición de memoria (estructura persistente).

6.5 El contenedor `map<K, V>`

La STL de C++ ofrece el contenedor `map<K, V>`, un diccionario implementado internamente como un **BST equilibrado** (árbol Rojo-Negro). Asocia **claves únicas** (K) a **valores** (V).

```
#include <map>
using namespace std;

map<string, int> m;
```

```
m["uno"] = 1;
m["diez"] = 10;
```

Internamente, cada elemento es un `pair<K, V>` con los campos `first` (clave) y `second` (valor).

El operador `[]`: la puerta de entrada

El operador `[]` es la forma más natural de usar un `map`. Si la clave **no existe**, la crea automáticamente con el valor por defecto del tipo (0 para `int`, "" para `string`).

Usar `m["clave"]` en un `map` `const` es **error de compilación** porque no puede crear elementos. Utiliza `m.find("clave")` o `m.at("clave")` en contextos `const`.

`find`: la búsqueda segura

```
map<string, int> m = {{"uno", 1}, {"diez", 10}};

auto it = m.find("diez");
if (it != m.end()) {
    cout << "Valor asociado: " << it->second << endl; // 10
}
```

`find` devuelve un **iterador** apuntando al par {clave, valor} si lo encuentra, o a `m.end()` si no está. Coste: $\mathcal{O}(\log n)$.

`insert`: añadir sin sobrescribir

```
m.insert({"veinte", 20});
m.insert({"veinte", -20}); // ;NO sustituye si ya existe!
```

6.6 El `map` como acumulador

Uno de los usos estrella del `map` es acumular y contar. El operador `[]` hace todo el trabajo:

1. Si la clave **no existe** $\rightarrow$ la crea con valor 0, entonces hace `++`.
2. Si la clave **existe** $\rightarrow$ recupera el valor actual y hace `++`.

Ejemplo 1: Frecuencia de palabras

```
#include <map>
#include <iostream>
using namespace std;

int main() {
    map<string, int> word_count;
    string word;
    while (cin >> word) {
        word_count[word]++; // <-- ;Pura magia!
    }
    for (auto it = word_count.begin(); it != word_count.end(); ++it) {
        cout << it->first << ": " << it->second << endl;
    }
}
```

Ejemplo 2: Agrupa palabras por longitud

```
map<int, vector<string>> by_length;
string word;
while (cin >> word) {
    by_length[word.size()].push_back(word);
}
```

6.7 Iterar sobre un map

Los iteradores de `map` recorren los elementos **en orden ascendente por clave** (porque internamente es un BST ordenado). El operador `->` accede a los campos `first` y `second` del par:

```
map<string, int> m;
// ... (llenar el map)

for (auto it = m.begin(); it != m.end(); ++it) {
    cout << "clave: " << it->first
         << ", valor: " << it->second << endl;
}
```

O con el bucle `for-each` moderno (C++11):

```
for (const auto& [clave, valor] : m) {
    cout << clave << " → " << valor << endl;
}
```

6.8 El contenedor set<T>

Un `set<T>` es un `map` donde solo existe la clave, sin valor asociado. Se utiliza para: - **Eliminar duplicados** de una secuencia. - **Contar el vocabulario** (palabras únicas). - **Comprobar pertenencia** en $\mathcal{O}(\log n)$.

```
#include <set>
using namespace std;

set<string> vocabulario;
string palabra;
while (cin >> palabra) {
    vocabulario.insert(palabra);
}
cout << "Palabras únicas: " << vocabulario.size() << endl;
```

Nota Principal

Si necesitas claves **repetidas** (multiset o multimap), C++ ofrece `multiset<T>` y `multimap<K, V>`. En un `map` normal, dos inserciones con la misma clave **no** añaden un segundo elemento.

6.9 Cuándo usar cada contenedor

Situación	Contenedor recomendado
Búsqueda frecuente en datos ordenados	<code>map<K,V></code> o <code>set<T></code>
Sin orden, búsqueda a máxima velocidad	<code>unordered_map<K,V></code> ($\mathcal{O}(1)$ amortizado)

Situación	Contenedor recomendado
Acceso por índice numérico	<code>vector<T></code>
Inserción/borrado frecuente en medio	<code>list<T></code>
Colección de elementos únicos	<code>set<T></code>
Acumulador clave $\rightarrow$ contador/lista	<code>map<K, vector<T>></code>